

BLOCO 5 • SISTEMAS OPERACIONAIS DISTRIBUÍDOS

Sistemas Operacionais Distribuídos

Conceitos e Arquitetura

Aula 24 • Sistemas Distribuídos • Uni-FACEF

Prof. Leandro Borges

Agenda de hoje

01

O que é um Sistema Operacional Distribuído (SOD)

02

SO de Rede (NOS) x SO Distribuído (DOS)

03

Modelos de arquitetura: workstation, processor pool, híbrido

04

O papel do middleware

05

Desafios de projeto de um SOD

06

Próxima aula e referências

Retomando: a ilusão de um único sistema

Desde a Semana 2 vimos que o objetivo de um sistema distribuído é a transparência: esconder do usuário e do programador que existem várias máquinas separadas por trás do serviço. Um Sistema Operacional Distribuído (SOD) é a tentativa mais ambiciosa de entregar essa ilusão — não apenas comunicação transparente, mas um SO inteiro que parece rodar numa única máquina virtual gigante.

1

SO tradicional

Gerencia processos, memória e arquivos em UMA máquina, com um único kernel no comando.

2

Rede de computadores

Várias máquinas independentes, conectadas, cada uma com seu próprio SO e decisões locais.

3

SO Distribuído (SOD)

Estende as funções de gerência do SO tradicional através de várias máquinas, como se fosse uma só.

SO de Rede (NOS) x SO Distribuído (DOS)

SO de Rede (NOS)

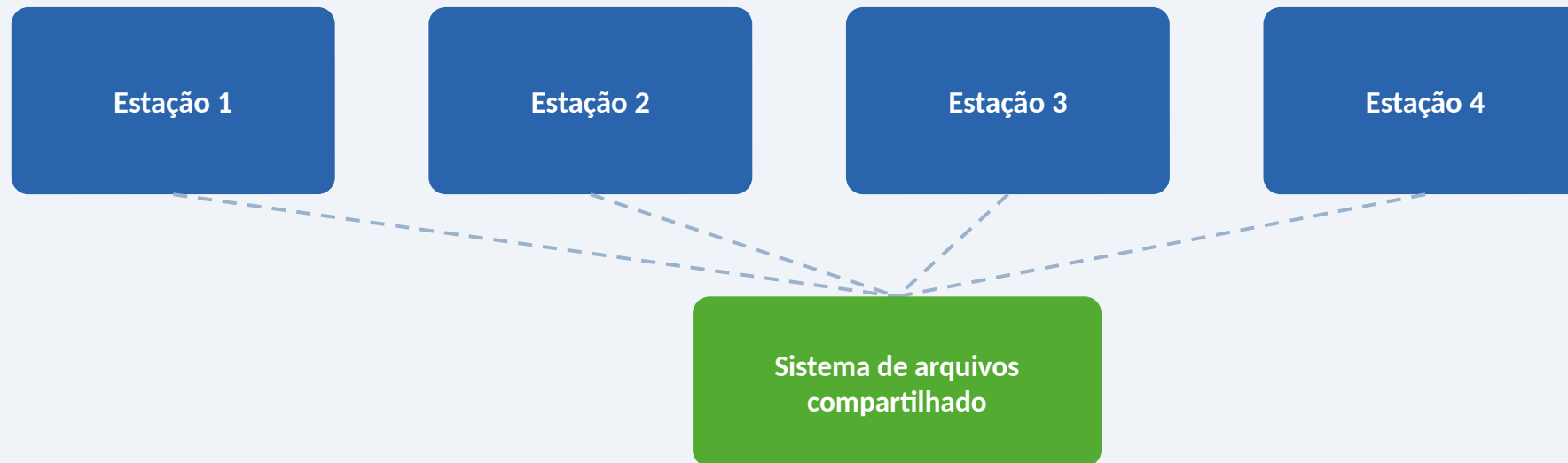
- Cada máquina roda seu próprio SO, de forma independente e autônoma.
- Serviços de rede (compartilhamento de arquivos, impressão, login remoto) são adicionados como camada sobre esse SO local.
- O usuário sabe que existem várias máquinas — precisa indicar em qual delas quer rodar algo ou acessar um recurso.
- Baixa transparência, mas alta autonomia e simplicidade de administração.

SO Distribuído (DOS)

- Uma única camada de software (kernel distribuído) gerencia todos os recursos de todas as máquinas de forma unificada.
- O usuário não sabe (nem precisa saber) em qual máquina física seus processos ou arquivos estão.
- Escalonamento, memória e arquivos são decididos globalmente, não máquina a máquina.
- Alta transparência, mas muito mais difícil de projetar e implementar — por isso é raro na prática.

Modelos de arquitetura (1/3): Workstation Model

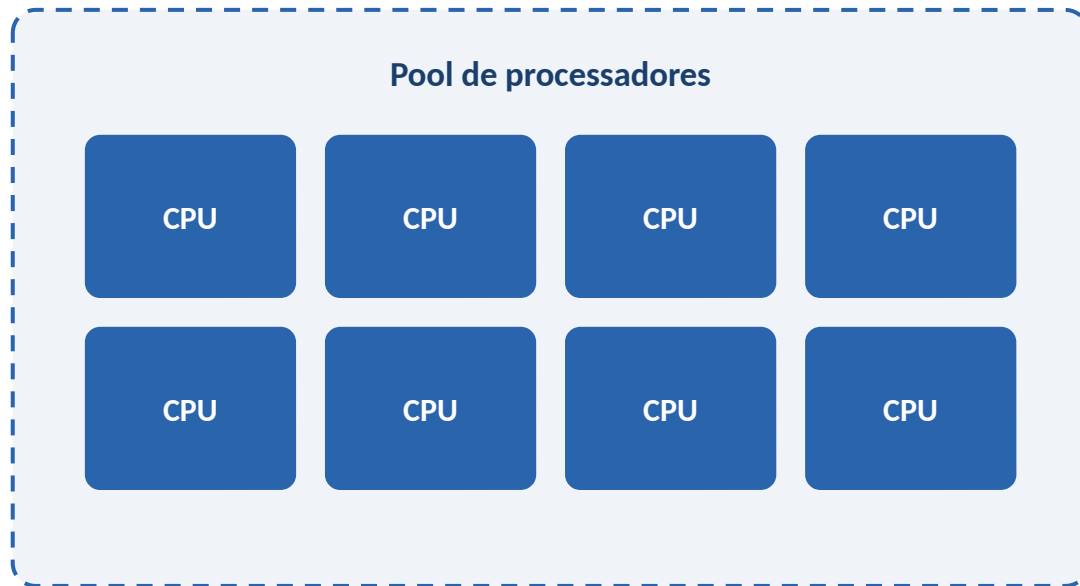
Cada usuário tem sua própria estação de trabalho (workstation), com CPU e memória dedicadas. As máquinas ficam ociosas boa parte do tempo, então compartilham um sistema de arquivos distribuído em rede — o modelo mais comum em laboratórios e escritórios.



Cada máquina mantém seu próprio SO local; a distribuição fica restrita ao acesso a arquivos e recursos remotos.

Modelos de arquitetura (2/3): Processor Pool Model

Em vez de processadores fixos por usuário, existe um banco (pool) compartilhado de processadores, sem dono definido. Quando um usuário submete uma tarefa, o sistema aloca dinamicamente um ou vários processadores do pool para executá-la, e os libera ao final.



Uso mais eficiente da capacidade de processamento total (nada fica ocioso por longos períodos).



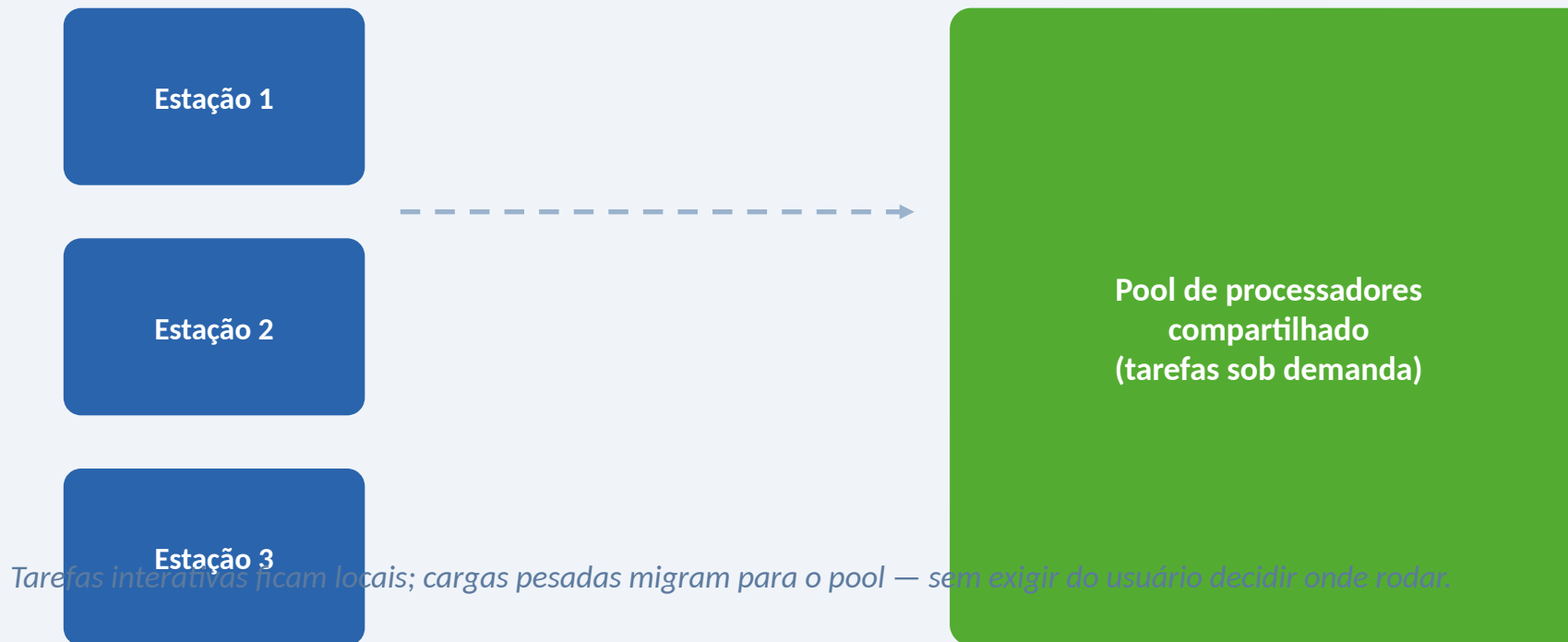
Escalonamento global decide onde cada tarefa roda, conforme carga e disponibilidade.



Menos intuitivo para o usuário: não existe 'minha máquina' fixa.

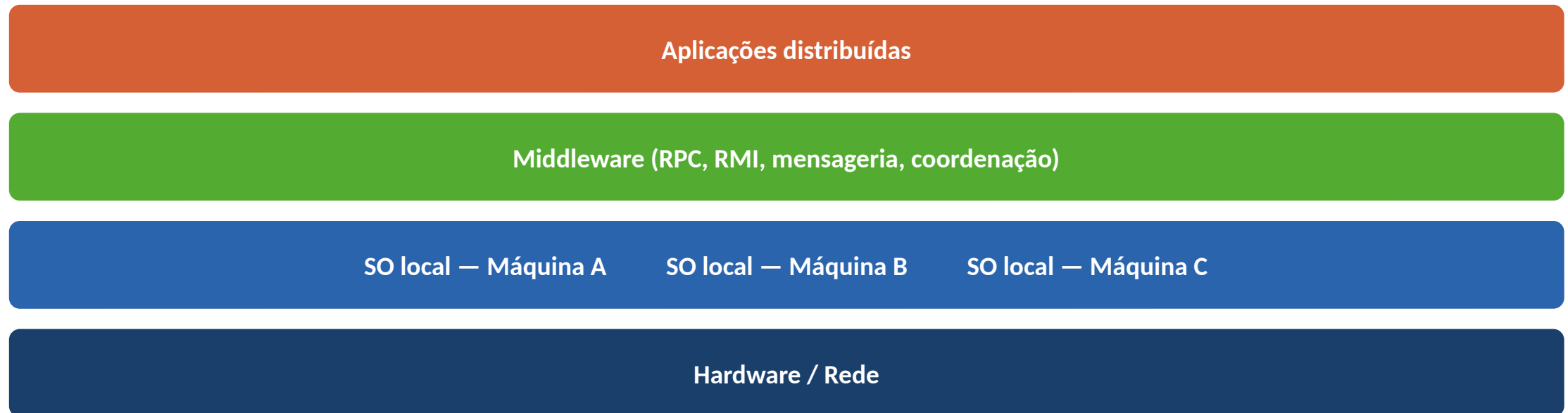
Modelos de arquitetura (3/3): Modelo Híbrido

Combina os dois modelos anteriores: cada usuário tem uma workstation pessoal para tarefas interativas do dia a dia, mas também existe um pool de processadores compartilhado para tarefas pesadas ou paralelas, submetidas sob demanda. É o equilíbrio mais comum entre desempenho, custo e experiência do usuário.



O papel do middleware

SODs "puros" exigem um kernel distribuído único, complexo demais para ser adotado em larga escala. Na prática, a indústria resolve o mesmo problema com middleware: uma camada de software que roda sobre SOs convencionais (cada máquina com seu próprio Linux, Windows etc.) e simula, para as aplicações, a transparência de um sistema único — sem reescrever o kernel.



Por isso o middleware — não o SOD puro — é a abordagem dominante em sistemas distribuídos reais.

Desafios de projeto de um SOD

1

Gerência global de recursos

Decidir onde alocar CPU, memória e arquivos entre todas as máquinas, não apenas localmente.

2

Escalonamento distribuído

Escolher em qual nó cada processo roda, balanceando carga sem visão centralizada de baixa latência.

3

Comunicação entre kernels

Coordenar múltiplos kernels via rede — mais lenta e menos confiável que comunicação dentro de uma máquina.

Síntese: do SO local ao SOD

	SO tradicional	SO de Rede (NOS)	SO Distribuído (DOS)
Escopo da gerência	Uma máquina	Múltiplas máquinas, autônomas	Múltiplas máquinas, unificadas
Transparência	N/A	Baixa — usuário escolhe a máquina	Alta — ilusão de máquina única
Exemplo	Linux, Windows	NOS (compartilhamento de rede)	SOD puro (raro) ou middleware

PRÓXIMA AULA

Aula 25 — SO Distribuídos: Gerência de Processos e Escalonamento Distribuído

Vamos aprofundar como um SOD decide onde cada processo executa: migração de processos, algoritmos de escalonamento distribuído e balanceamento de carga entre nós.

Terça-feira, 10/11

Referências

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems: Principles and Paradigms. 3. ed. Pearson, 2017. Cap. 1 (Introdução) e Cap. 3 (Processos) — arquitetura de sistemas operacionais distribuídos.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. Distributed Systems: Concepts and Design. 5. ed. Addison-Wesley, 2011. Cap. 1 e 2 — modelos de sistema e arquitetura.

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems. 4. ed. 2023 (edição digital gratuita) — capítulos sobre arquitetura e middleware.